TP2 – Les fondations du projet

React Native · En trinôme · Projet, jalon 1

Votre sujet est choisi, votre dépôt commun existe. Ce TP pose l'**architecture MVC** de votre projet : tout ce que vous ajouterez ensuite (navigation, API, Firebase) viendra s'y brancher sans tout réécrire.

Ce TP en bref

- **Objectifs** : organiser le code en modèles, services, hooks contrôleurs et vues, typer les données, gérer un chargement asynchrone et partager un état global.
- En trinôme, dans le **dépôt du projet**. Ce jalon n'est pas noté : il pose les bases de votre rendu final.
- Le cours 2 (catalogue de séjours) vous sert de **modèle** : appliquez la même structure à **votre** sujet.
- L'IA est autorisée, à une condition : chaque membre doit pouvoir expliquer chaque fichier.

0 Le dépôt et l'équipe

Git · organisation

1. Un seul membre crée le projet Expo (`npx create-expo-app@latest` , puis `npm run reset-project` en répondant `n` pour supprimer l'exemple, et `"output": "single"` dans `app.json`), le pousse sur le dépôt `rn-projet-nom1-nom2-nom3` , puis invite les deux autres et **Chris-CHEVALIER**, si ce n'est pas déjà fait.
2. Les deux autres clonent le dépôt et lancent `npm install` puis `npx expo start` .
3. Répartissez les **trois parties** du projet (A · Catalogue, B · Navigation et compte, C · Données partagées : voir le sujet du projet). Créez le `README.md` : le sujet en trois phrases, l'API publique envisagée, les membres (prénom, nom, compte GitHub) et le **propriétaire de chaque partie**.

> **Travailler à trois sans se marcher dessus**
>
> Chacun travaille sur sa partie : **A** sur les étapes 1 à 4 (modèle, service, hooks et catalogue), **B** sur le modèle et le service de votre première collection, **C** sur l'étape 5 (Context et composant qui le lit). Faites des commits courts et faites `git pull` avant chaque `git push` . Chacun commite avec son propre compte.

1 Le modèle

TypeScript · fonctions pures

Dans `src/models/`, un fichier par entité principale de votre sujet (par exemple `recipe.ts`).

1. Écrivez le **type** de votre entité : au moins six propriétés, dont un identifiant, une union de valeurs (une catégorie, un statut…) et une propriété facultative.
2. Écrivez le type de ses **filtres** : au moins une recherche textuelle et un filtre par catégorie.
3. Écrivez une **fonction pure** `matchesFilters(item, filters)` qui renvoie un booléen.

Dans `src/data/`, créez **au moins 10 éléments** réalistes. Ils seront remplacés par l'API au TP3.

> **Partie B**
>
> Sur le même modèle, écrivez le type, les données factices et le service de votre **première collection** Firebase (par exemple vos recettes personnelles). Firebase la remplacera au TP4.

2 Le service

async · Promise

Dans `src/services/`, écrivez une fonction asynchrone qui renvoie vos données, par exemple `fetchRecipes(): Promise<Recipe[]>`.

- Elle attend **800 ms** avant de répondre, pour simuler le réseau.
- Ajoutez temporairement une ligne qui **lève une erreur** : vous en aurez besoin pour tester l'étape 4. Retirez-la ensuite.

Indice : simuler une attente

Simuler le réseau

```
const wait = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
```

3 Les hooks contrôleurs

Hooks personnalisés · union discriminée

1. Dans `src/hooks/`, écrivez le hook qui charge vos données (par exemple `useRecipes()`).
   - Son état est une **union discriminée** à trois cas : `"loading"`, `"error"` (avec un message) et `"success"` (avec les données).
   - Il renvoie cet état et une fonction `reload`.
2. Écrivez le hook des filtres (par exemple `useRecipeFilters(items)`). Il renvoie les filtres, une fonction pour les modifier et la liste filtrée, **calculée** avec `matchesFilters`.

> La liste filtrée n'est pas un état. Si vous écrivez `useState` pour elle, c'est qu'une donnée est stockée deux fois.

4 Les vues et l'écran

Composants purs · responsive

1. Dans `src/components/`, écrivez au moins **trois composants purs** : la carte d'un élément, la barre de filtres et un message d'état (chargement, erreur avec un bouton « Réessayer »).
2. Dans `src/app/index.tsx`, l'écran **assemble** les hooks et les composants. Il affiche :
   - un indicateur pendant le chargement ;
   - un message et un bouton « Réessayer » en cas d'erreur (testez-le avec l'erreur de l'étape 2) ;
   - le catalogue responsive (grille sur ordinateur, liste sur téléphone), avec un message quand aucun élément ne correspond.

> **Vérifiez vos vues**
>
> Aucun fichier de `src/components/` ne doit importer un service, appeler `fetch` ou contenir un `useEffect` qui charge des données.

5 Un état global

Context · Provider

1. Choisissez une donnée que **plusieurs composants** partageront : des favoris, une sélection, des préférences d'affichage…
2. Dans `src/context/`, créez le Context typé et son Provider. Dans `src/hooks/`, écrivez le hook d'accès, qui lève une erreur hors du Provider.
3. Placez le Provider dans `src/app/_layout.tsx`.
4. Dans `src/components/`, écrivez un composant qui lit cet état **sans recevoir de props** (un badge « ★ 3 favoris », par exemple). Affichez-le dans l'en-tête du catalogue.

✓ Avant de rendre

- ☐ L'arborescence contient `models/`, `data/`, `services/`, `hooks/`, `context/`, `components/` et `app/`.
- ☐ `npx tsc --noEmit` n'affiche aucune erreur.
- ☐ Les trois états (chargement, erreur, succès) s'affichent correctement.
- ☐ Le catalogue est une grille sur ordinateur et une liste sur téléphone.
- ☐ Une modification de l'état global se voit aussitôt dans le composant qui le lit.
- ☐ Le README indique le propriétaire de chaque partie, et chacun a commité sur la sienne.

> **Rendu**
>
> Poussez votre travail en fin de séance. Je relis l'état du dépôt à la fin de la journée : pas besoin d'e-mail, votre invitation me donne accès au dépôt.